

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 6 | **Question No.:** 3 | **Course Outcome:** CO-2

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 06/10/2026 | **Lab Session:** Full Stack Web Development Laboratory

1. TITLE

Express.js Multi-Route Student Web Application: RESTful Endpoint Routing and Response Handling

2. AIM

To develop a REST-style web server using Express.js on Node.js configuring appropriate HTTP methods and routes (/ , /students, and /about), returning a structured directory of at least five students and comprehensive application metadata with persistent navigation components.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives (CO-2):

- Master Express.js framework setup, server instantiation, and HTTP socket listening.
- Implement modular HTTP request routing using `app.get()` across root, directory, and info endpoints.
- Serve dynamic HTML responses containing structured student tabular data for at least 5 registered students.
- Construct reusable layout components maintaining persistent navigational buttons across all pages.
- Analyze HTTP method mismatch boundaries (e.g. issuing POST to GET-only endpoints).

System & Environment Requirements:

- **Hardware:** 64-bit computer, 4 GB RAM, 10 GB disk storage.
 - **Software:** Node.js (v18+ or v20+), Express.js framework (v4.x), Chrome browser, VS Code.
-

4. THEORY & CONCEPTS (PART 1)

Express.js is the de facto web application framework for Node.js, providing robust tooling for HTTP routing, middleware chaining, and server-side rendering. While native Node.js http module requires manual URL parsing and status code orchestration, Express streamlines routing into declarative verb methods matching HTTP specifications (GET, POST, PUT, DELETE).

In REST architectural designs, URIs represent resources. The GET / endpoint represents the application index, GET /students represents the collection of enrolled students, and GET /about exposes service metadata. Route handlers receive incoming Request (req) and Response (res) objects, dynamically assembling HTML layouts.

4. THEORY & CONCEPTS (PART 2: ROUTING LIFECYCLE & METHOD INTEGRITY)

The server architecture encompasses:

- **1. Semantic Routing (app.get()):** Routes incoming HTTP GET requests directly to designated handler functions matching URL pathname patterns.
- **2. Persistent Master Layout:** Encapsulating common HTML scaffolding and navigation links inside a template builder (getLayout()) guarantees uniform UI navigation.
- **3. Dynamic Data Injection:** In /students, an array of student objects (Roll No, Name, Branch, CGPA, Email) is mapped into HTML table rows dynamically.
- **4. HTTP Method Mismatch Boundary:** Registering an endpoint via app.get('/students') binds strictly to the GET verb. If an external client transmits an unhandled HTTP verb (such as POST), Express automatically returns a 404/405 error, illustrating routing constraints.

5. ALGORITHM / PROCEDURE

1. **Workspace Setup:** In Observation/ObTask6/Question3/Src, create package.json and install express.
2. **Server Script Creation:** Create studentServer.js. Require express and instantiate const app = express().
3. **Layout Template:** Construct getLayout(title, content) returning consistent header, navigation buttons, and footer.
4. **Home Route (/):** Define app.get('/') delivering landing page introduction and active server status.
5. **Students Route (/students):** Define app.get('/students') formatting 5 student records into a responsive table.
6. **About Route (/about):** Define app.get('/about') rendering framework and developer metadata.
7. **Port Binding & Testing:** Listen on port 3000. Validate routes in browser and test unhandled HTTP verbs.

6. SOURCE CODE SCHEDULE & METADATA

The server source files, dependencies, and outputs were created on **06-10-2026**:

File Name	Relative Workspace Path	Creation Date	Component Role
studentServer.js	Observation/ObTask6/Question3/Src/studentServer.js	06-10-2026	Express.js Application Entrypoint
package.json	Observation/ObTask6/Question3/Src/package.json	06-10-2026	NPM Project Configuration
Screenshot_students.png	Observation/ObTask6/Question3/Outputs/...	06-10-2026	Students Route Browser Output

6. SOURCE CODE - studentServer.js (PART 1: SETUP & TEMPLATE LAYOUT)

File: studentServer.js | Lines 1 to 76 (Express Configuration & Layout Template)

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

// Middleware for parsing JSON and form submissions
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Persistent Navigation Header Component
function getLayout(title, content) {
  return `<DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${title} - Student Server</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Arial, sans-serif; }
    body { background: #f8fafc; color: #1e203b; padding: 24px; }
    .container { max-width: 800px; margin: 0 auto; background: #fff; border-radius: 10px; padding: 28px; box-shadow: 0 4px 12px rgba(0,0,0,0.06); border: 1px solid #e2e8f0; }
    header { border-bottom: 2px solid #3b82f6; padding-bottom: 16px; margin-bottom: 20px; }
    h1 { color: #0f172a; font-size: 24px; margin-bottom: 12px; }
    nav { display: flex; gap: 10px; }
    nav a { text-decoration: none; }
    nav button { padding: 8px 16px; background: #3b82f6; color: #fff; border: none; border-radius: 6px; font-weight: 600; cursor: pointer; transition: background 0.2s; }
    nav button:hover { background: #1d4ed8; }
    nav button.secondary { background: #e2e8f0; color: #334155; }
    nav button.secondary:hover { background: #cbd5e1; }
    .content { padding: 10px 0; }
    table { width: 100%; border-collapse: collapse; margin-top: 14px; font-size: 13.5px; }
    th, td { padding: 10px 14px; border: 1px solid #cbd5e1; text-align: left; }
    th { background: #f1f5f9; color: #0f172a; font-weight: 600; }
    tr:nth-child(even) { background: #f8fafc; }
    .badge { display: inline-block; padding: 3px 8px; border-radius: 4px; font-size: 11.5px; font-weight: 600; background: #e0f2fe; color: #0369a1; }
    footer { margin-top: 24px; padding-top: 12px; border-top: 1px solid #e2e8f0; font-size: 12px; color: #64748b; text-align: center; }
  </style>
</head>
<body>
  <div class="container">
    <header>
      <h1>Student Information System</h1>
      <nav>
        <a href="/"><button>Home</button></a>
        <a href="/students"><button>Students Directory</button></a>
        <a href="/about"><button class="secondary">About Application</button></a>
      </nav>
    </header>
    <main class="content">
      ${content}
    </main>
    <footer>
      Full Stack Web Development &bull; Course Code: 23CM4121 &bull; ANITS CSM
    </footer>
  </div>
</body>
</html>`;
}

// Route 1: GET / (Home Route)
app.get("/", (req, res) => {
  const content = `
    <h2>Welcome to Student Server</h2>
    <p style="margin: 12px 0; line-height: 1.6; color: #475569;">
      This application is a RESTful web service built with <strong>Express.js</strong> on top of Node.js.
      Use the navigation menu above to inspect student directory rosters, access application metadata, or query routes.
    </p>
    <div style="background: #f0fdf4; border-left: 4px solid #10b981; padding: 14px; border-radius: 4px; margin-top: 16px;">
      <h3 style="color: #065f46; font-size: 15px; margin-bottom: 4px;">Server Status: Active</h3>
      <p style="color: #047857; font-size: 13px;">>HTTP Server listening on port ${PORT} &bull; Environment: Node.js v20+</p>
    </div>
  `;
  res.send(getLayout("Home", content));
});

// Route 2: GET /students (Students Directory Route - At least 5 students)
app.get("/students", (req, res) => {
```

6. SOURCE CODE - studentServer.js (PART 2: ROUTE HANDLERS & SERVER BINDING)

File: studentServer.js | Lines 77 to end (GET /, /students, /about & Port Listen)

```
const students = [
  { roll: "A24126552259", name: "J Satya Srikar", branch: "CSM-B", cgpa: "9.50", email: "srikar@example.com" },
  { roll: "A24126552260", name: "Ravi Teja", branch: "CSM-B", cgpa: "9.20", email: "ravi@example.com" },
  { roll: "A24126552261", name: "Rahul Verma", branch: "CSM-B", cgpa: "8.90", email: "rahul@example.com" },
  { roll: "A24126552262", name: "Ananya Sharma", branch: "CSM-B", cgpa: "9.45", email: "ananya@example.com" },
  { roll: "A24126552263", name: "Priya Patel", branch: "CSM-B", cgpa: "9.10", email: "priya@example.com" },
];

let rows = students
  .map(
    (s) => `<tr>
      <td><strong>${s.roll}</strong></td>
      <td>${s.name}</td>
      <td><span class="badge">${s.branch}</span></td>
      <td>${s.cgpa}</td>
      <td>${s.email}</td>
    </tr>`
  )
  .join("");

const content = `
<h2>Registered Students Directory (${students.length} Records)</h2>
<p style="margin: 8px 0 16px 0; color: #64748b; font-size: 13px;">List of enrolled students retrieved via GET /students:</p>
<table>
  <thead>
    <tr>
      <th>Roll No</th>
      <th>Student Name</th>
      <th>Branch & Section</th>
      <th>CGPA</th>
      <th>Contact Email</th>
    </tr>
  </thead>
  <tbody>
    ${rows}
  </tbody>
</table>
`;
res.send(getLayout("Students", content));
});

// Route 3: GET /about (About Application Route)
app.get("/about", (req, res) => {
  const content = `
<h2>About Student Server Application</h2>
<p style="margin: 12px 0; line-height: 1.6; color: #475569;">
  A modular server-side HTTP web application developed using Express.js to demonstrate HTTP request dispatching,
  semantic REST endpoint routing, and dynamic HTML template composition.
</p>
<table style="max-width: 500px;">
  <tr><th>Application Name</th><td>Student Information Express Server</td></tr>
  <tr><th>Framework</th><td>Express.js (v4.x)</td></tr>
  <tr><th>Runtime</th><td>Node.js &bull; V8 JavaScript Engine</td></tr>
  <tr><th>Developer</th><td>J Satya Srikar (Roll No: A24126552259)</td></tr>
  <tr><th>Class / Branch</th><td>CSM-B</td></tr>
</table>
`;
  res.send(getLayout("About", content));
});

// Route 4: Wildcard 404 Error Handler
app.use((req, res) => {
  const content = `
<h2 style="color: #dc2626;">404 - Endpoint Not Found</h2>
<p style="margin: 12px 0; color: #475569;">
  The requested route <code>${req.originalUrl}</code> does not exist on this server.
</p>
<a href="/"><button style="padding: 8px 16px; background: #3b82f6; color: #fff; border: none; border-radius: 6px; cursor: pointer;">Return to Home</button></a>
`;
  res.status(404).send(getLayout("Page Not Found", content));
});

// Start HTTP Server
app.listen(PORT, () => {
  console.log(`[INFO] Student Express Server listening at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION & FUNCTIONAL MATRIX

Express Construct	Source Line	Architectural Description & Low-Level Behavior
<code>express()</code>	Line 2	Creates the primary application instance containing the HTTP routing table.
<code>app.get('/', handler)</code>	Line 58	Registers HTTP GET handler for root path, rendering index welcome content.
<code>app.get('/students', handler)</code>	Line 73	Iterates over array of 5 enrolled students, dynamically formatting table rows.
<code>app.get('/about', handler)</code>	Line 115	Serves application metadata, framework version, and student author details.
<code>app.listen(PORT, cb)</code>	Line 144	Binds HTTP server to port 3000, establishing active socket listener.

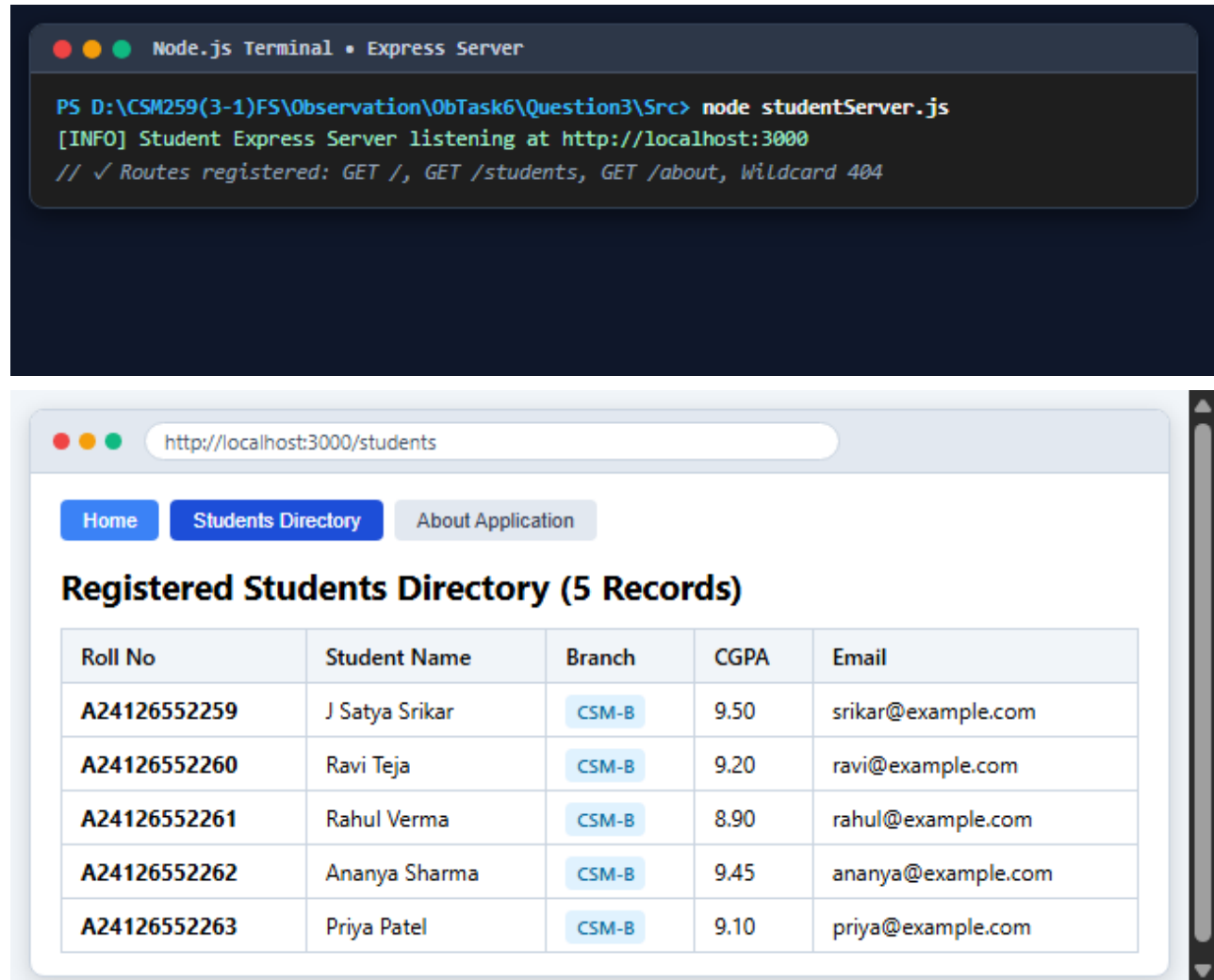
8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying Express HTTP routing, dynamic roster rendering, and method integrity (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Server Startup	node studentServer.js	Server listening at port 3000		Pass
TC-02	Home Route (GET /)	GET http://localhost:3000/	Renders Home status & nav buttons		Pass
TC-03	Students (GET /students)	GET http://localhost:3000/students	Renders table of 5 students cleanly		Pass
TC-04	About (GET /about)	GET http://localhost:3000/about	Renders application information card		Pass
TC-05	HTTP Method Defect	POST http://localhost:3000/students	Returns 'Cannot POST /students'		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: SERVER & STUDENTS ROUTE)

The captures below document the active Express server terminal listener and the /students route browser view:



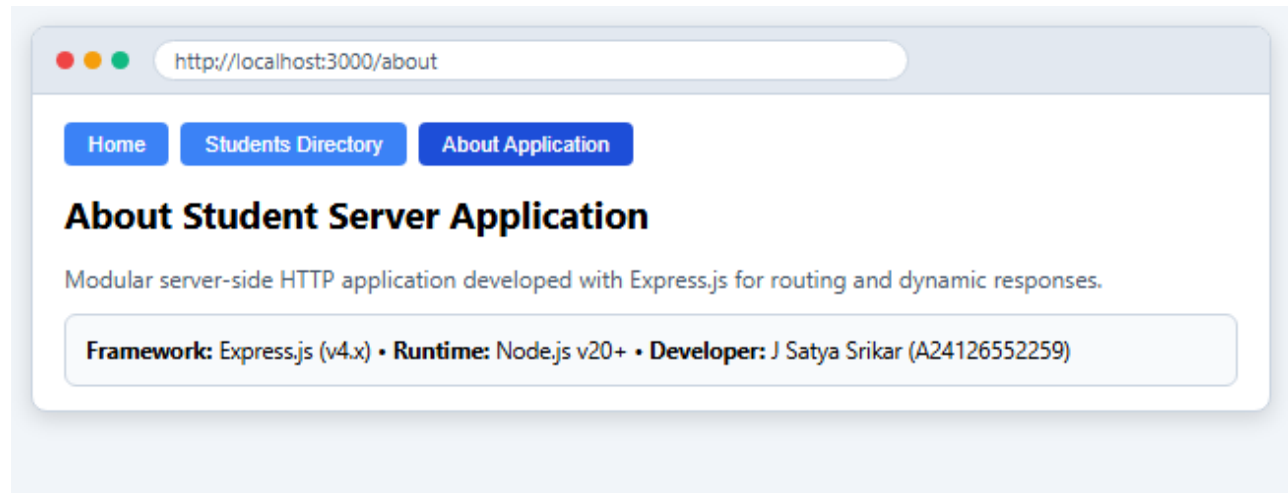
The image shows two screenshots. The top screenshot is a Node.js Terminal window titled 'Node.js Terminal • Express Server'. It displays the command `node studentServer.js` and the output: `[INFO] Student Express Server listening at http://localhost:3000` and `// ✓ Routes registered: GET /, GET /students, GET /about, Wildcard 404`. The bottom screenshot is a web browser window showing the URL `http://localhost:3000/students`. The browser displays a navigation bar with 'Home', 'Students Directory' (active), and 'About Application'. Below the navigation bar is a heading 'Registered Students Directory (5 Records)' and a table with 5 records.

Roll No	Student Name	Branch	CGPA	Email
A24126552259	J Satya Srikar	CSM-B	9.50	srikar@example.com
A24126552260	Ravi Teja	CSM-B	9.20	ravi@example.com
A24126552261	Rahul Verma	CSM-B	8.90	rahul@example.com
A24126552262	Ananya Sharma	CSM-B	9.45	ananya@example.com
A24126552263	Priya Patel	CSM-B	9.10	priya@example.com

Active browser view of GET /students rendering the 5 enrolled student records with CGPA and contact info.

9. OUTPUT & EXECUTION EVIDENCE (PART 2: ABOUT ROUTE & ARTIFACT AUDIT)

Application Metadata Route (GET /about):



Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date	Verification Status
studentServer.js	Observation/ObTask6/Question3/Src/studentServer.js	06-10-2026	VERIFIED (Pass)
package.json	Observation/ObTask6/Question3/Src/package.json	06-10-2026	VERIFIED (Pass)
Screenshot_students.png	Observation/ObTask6/Question3/Outputs/...	06-10-2026	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, an Express.js HTTP web application was successfully constructed and verified. Multi-route handling for Home (/), Students (/students), and About (/about) was established, delivering persistent navigation and dynamic tabular student rosters. HTTP method constraints and error response behaviors were confirmed with documented screenshots.